

Multiway partitioning

Direct multiway partitioning assigns nodes to k parts without forcing a binary tree of cuts

The idea

- Multiway objectives count every edge that leaves its part
- Balance becomes a k-way size vector
- Teaching point

Pseudocode

- Multiway compares two constructors in pseudocode
- Both must report the same cutsize metric
- Open this module's examples file and find the Pseudocode section
- That written sketch is what you implement on the implement track and what the browser

Algorithm sketch

- Recursive k equals three lands A B, C, and D E with cut eight
- Alphabetical round-robin shreds heavy edges and scores about eighteen

Algorithm sketch — try these

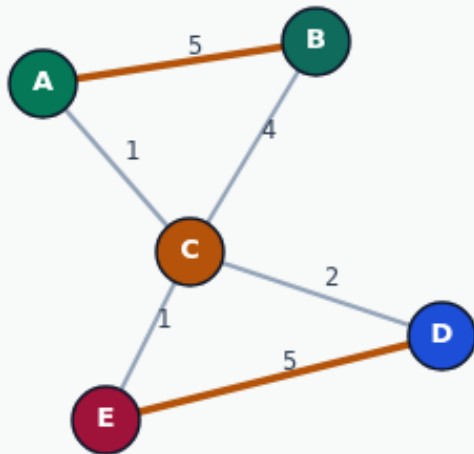
```
INPUT: G, k
OUTPUT: k-way assignment + cutsize
method A: recursive_bisection(G,k)
method B: round-robin / block assign labels
cut  $\leftarrow \sum w$  where  $side[u] \neq side[v]$ 
GOLDEN k=3 recursive: AB|C|DE cut=8
round-robin alphabetic: cut $\approx$ 18 (worse)
```

k-way vs naive assignment

MULTIWAY PARTITION

k-way vs naive assignment

Step 1 / 5 · problem · module03-01-multiway-partition



Explanation

Real placers need $k > 2$ parts. Recursive bisection respects graph structure; naive round-robin assigns nodes by index mod k — often catastrophic cut.

- $k=3$ on TINY_GRAPH
- Recursive: repeated bisection
- Round-robin: $A \rightarrow 0, B \rightarrow 1, C \rightarrow 2, \dots$

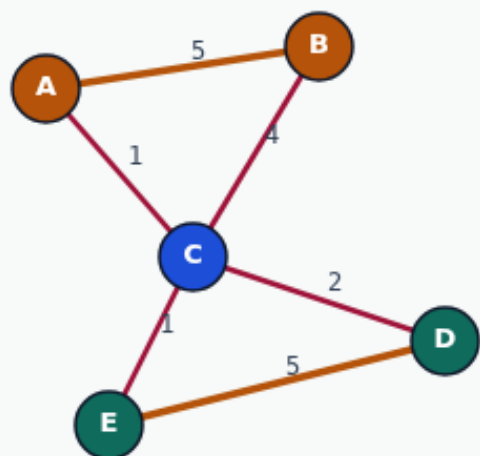
target $k=3$
same graph, two strategies

Recursive: AB|C|DE cut 8

MULTIWAY PARTITION

Recursive: AB|C|DE cut 8

Step 2 / 5 · recursive · module03-01-multiway-partition



Explanation

Recursive bisection to $k=3$ first splits ABC|DE (cut 3), then bisects ABC $\rightarrow$ AB|C|DE. A-B and D-E stay uncut; total cutsize is 8.

- Structure-aware splits
- A,B share a part
- D,E share a part

parts: AB|C|DE
cutsize: 8
3 labels

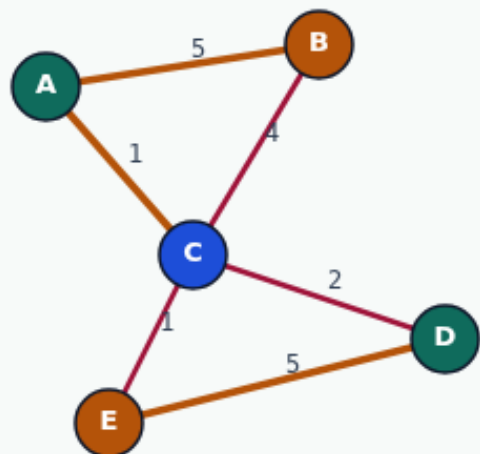
Recursive: AB|C|DE cut 8

Round-robin: AD|BE|C cut 18

MULTIWAY PARTITION

Round-robin: AD|BE|C cut 18

Step 3 / 5 · roundrobin · module03-01-multiway-partition



Explanation

Alphabetical round-robin puts A,D on part 0; B,E on part 1; C alone on part 2. Almost every edge crosses — cutsize 18.

- A and B on different parts
- D and E on different parts
- Ignores connectivity entirely

```
parts: AD|BE|C  
cutsize: 18  
gap vs recursive: 10
```

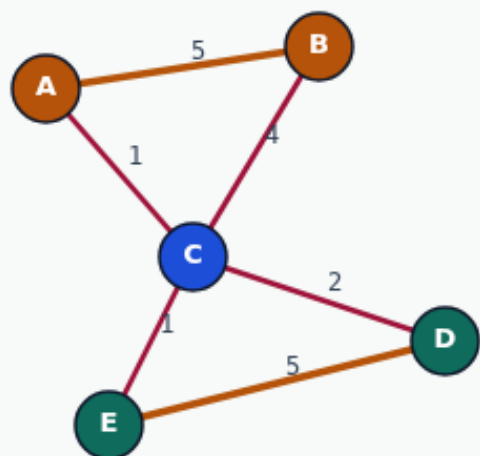
Round-robin: AD|BE|C cut 18

Structure beats indexing

MULTIWAY PARTITION

Structure beats indexing

Step 4 / 5 · compare · module03-01-multiway-partition



Explanation

Round-robin cuts A-B(5) and D-E(5) — the two strongest edges. Recursive keeps them internal and pays only bridge cuts through C.

- $18 - 8 = 10$ cut gap
- Same k , wildly different quality
- Why tools use recursive / multilevel

recursive: 8
round-robin: 18

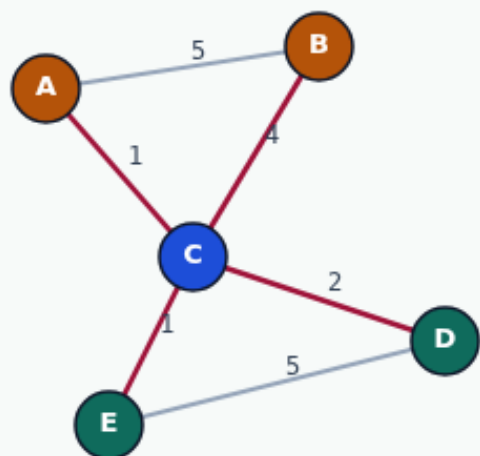
Structure beats indexing

Multiway literacy

MULTIWAY PARTITION

Multiway literacy

Step 5 / 5 · takeaway · module03-01-multiway-partition



Explanation

Direct k-way FM exists in production tools, but recursive bisection is the teaching baseline: global structure emerges from local 2-way splits.

- Never round-robin a netlist
- Recursive $k=3$: $AB|C|DE$
- Multilevel wraps the same idea

Starter golden: recursive cut=8

Browser lab track

- In the browser lab track, open the **multiway-partition** lab from the tools shelf
- Load the starter graph, run the algorithm once
- Work the challenges that lock the goldens

Implement track

- In the implement track
- Parse the tiny graph, run the algorithm with a deterministic seed
- Match the browser goldens before you claim the checklist

Pitfalls

- Common traps
- For multilevel flows, verify coarsening before you blame the refiner

Your turn

- Complete the checklist for at least one track, preferably both
- Implement until your metrics match the starter goldens
- When you're ready, take the short quiz, then continue to the next module